

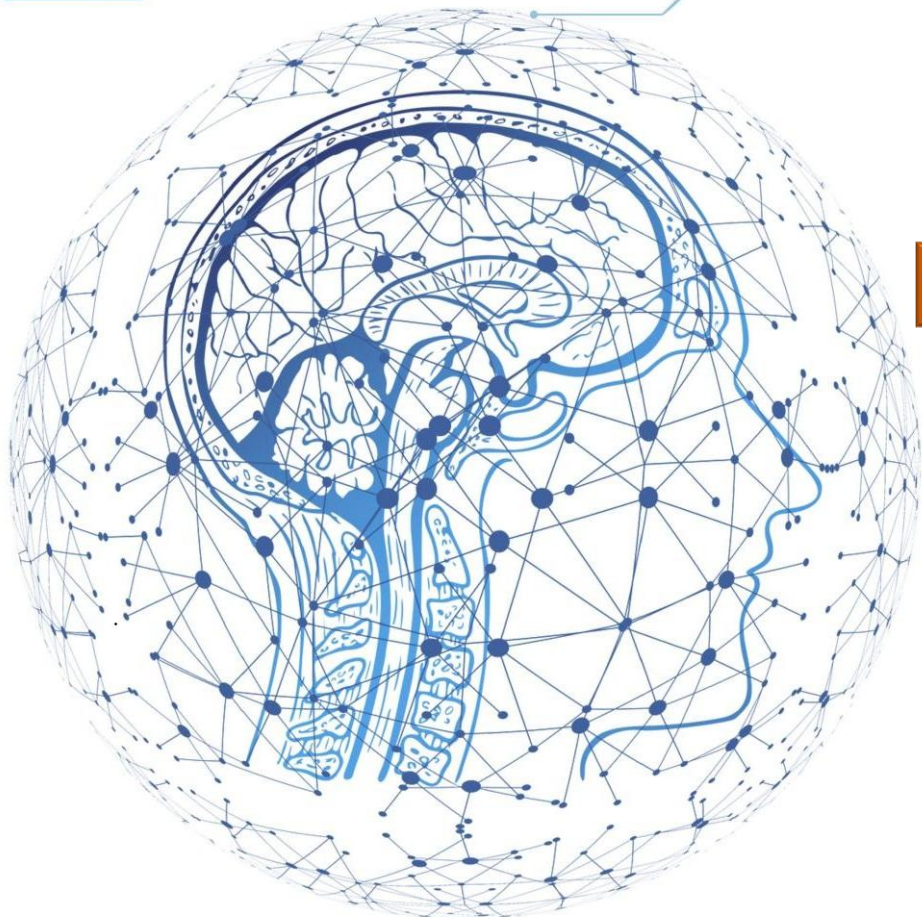
DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

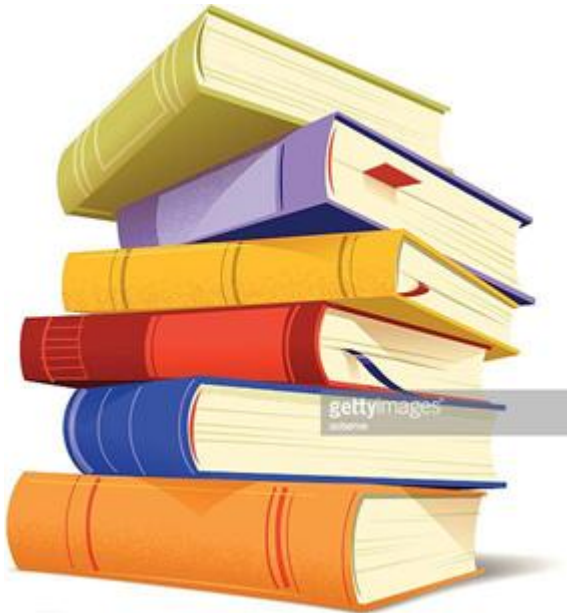
www.hoasen.edu.vn

Chương 4

Bắt đầu với mạng nơ-ron Phân loại và hồi quy



Giới thiệu Keras và TensorFlow



- Phân loại nhị phân với IMDB
- Phân loại đa lớp với Reuters
- Hồi quy với Boston Housing

Từ điển thuật ngữ quan trọng

- **Sample / Input:** một mẫu dữ liệu đầu vào
- **Prediction / Output:** đầu ra mô hình dự đoán
- **Target:** nhãn thật hoặc giá trị thật
- **Loss:** sai số giữa prediction và target
- **Class:** tập nhãn có thể có
- **Label:** nhãn cụ thể của một mẫu
- **Binary classification:** phân loại 2 lớp
- **Multiclass classification:** phân loại nhiều lớp
- **Scalar regression:** dự đoán một giá trị liên tục

Ba kiểu bài toán trong chương

- **IMDB (Internet Movie Database)**: phân loại review phim là tích cực hay tiêu cực
- **Reuters**: phân loại bản tin vào 1 trong 46 chủ đề
- **Boston Housing**: dự đoán giá nhà
- Điểm khác biệt cốt lõi:
 - Classification → output là nhãn
 - Regression → output là giá trị số liên tục

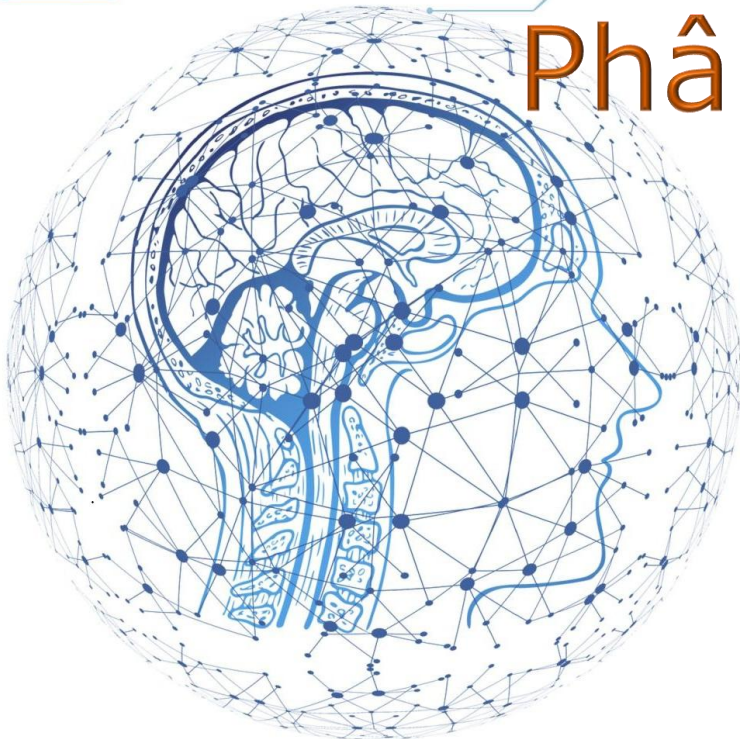
Workflow chung của một bài toán neural network

- Bước 1: Nạp dữ liệu
- Bước 2: Tiền xử lý dữ liệu
- Bước 3: Xây mô hình
- Bước 4: Compile với optimizer, loss, metrics
- Bước 5: Tách validation và train
- Bước 6: Huấn luyện, quan sát đường cong loss/accuracy
- Bước 7: Chọn mô hình tốt nhất
- Bước 8: Đánh giá trên test set
- Bước 9: Dự đoán trên dữ liệu mới

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Phân loại cảm xúc review phim (Binary Classification)



Phân loại cảm xúc review phim

- Đây là ví dụ về phân loại nhị phân
- Bài toán: sentiment analysis
- Input: văn bản review
- Output: 0 hoặc 1
- Mục tiêu: dự đoán review tích cực hay tiêu cực

IMDB dataset

- Bộ dữ liệu gồm **50,000** review phim
- Chia thành:
 - 25,000 mẫu train
 - 25,000 mẫu test
- Cân bằng:
 - 50% negative
 - 50% positive
- Dữ liệu đã được tiền xử lý sẵn:
 - mỗi review là một chuỗi số nguyên
 - mỗi số nguyên đại diện cho một từ

Giới hạn từ vựng và ý nghĩa

- Sử dụng:
 - `imdb.load_data(num_words=10000)`
- Chỉ giữ **10,000 từ xuất hiện thường xuyên nhất**
- Lợi ích:
 - giảm kích thước dữ liệu
 - giảm độ thưa
 - giúp bài toán dễ học hơn
- Từ hiếm bị loại bỏ vì ít đóng góp cho phân loại

Dữ liệu đầu vào

- train_data là list các review
- Mỗi review là list các chỉ số từ
- train_labels là list nhãn:
 - 0 = negative
 - 1 = positive
- Đây chưa phải dạng dữ liệu mà mô hình Dense có thể xử lý trực tiếp → Vì vậy ta cần đổi từ sequence sang vector

Tiền xử lý IMDB — vector hóa dữ liệu

- Mỗi review được mã hóa thành vector nhị phân kích thước 10,000
- Ý tưởng:
 - nếu từ thứ i xuất hiện trong review $\rightarrow$ vị trí $i = 1$
 - không xuất hiện $\rightarrow$ vị trí $i = 0$
- Đây là dạng **multi-hot encoding**
- Kết quả:
 - `x_train.shape = (25000, 10000)`

Kiến trúc mô hình cho binary classification

- Mô hình:
 - Dense(16, relu)
 - Dense(16, relu)
 - Dense(1, sigmoid)
- Vì sao dùng sigmoid ở đầu ra?
 - vì cần một xác suất từ 0 đến 1
- Vì sao dùng Dense + relu?
 - đơn giản, hiệu quả cho vector data

Compile mô hình

- Optimizer: `rmsprop`
- Loss: `binary_crossentropy`
- Metric: `accuracy`

Binary classification + sigmoid output → `binary_crossentropy`

Validation cho IMDB

- Tách 10,000 mẫu từ training set để làm validation
- Huấn luyện:
 - 20 epochs
 - batch size 512
- Theo dõi:
 - training loss
 - validation loss
 - training accuracy
 - validation accuracy

Đọc đồ thị loss và accuracy của IMDB

- Ban đầu:
 - training loss giảm
 - validation loss cũng giảm
- Sau vài epoch:
 - training loss tiếp tục giảm
 - validation loss tăng trở lại
- Đây là dấu hiệu **overfitting**: Validation accuracy đạt đỉnh rồi giảm

Dự đoán trên dữ liệu mới với IMDB

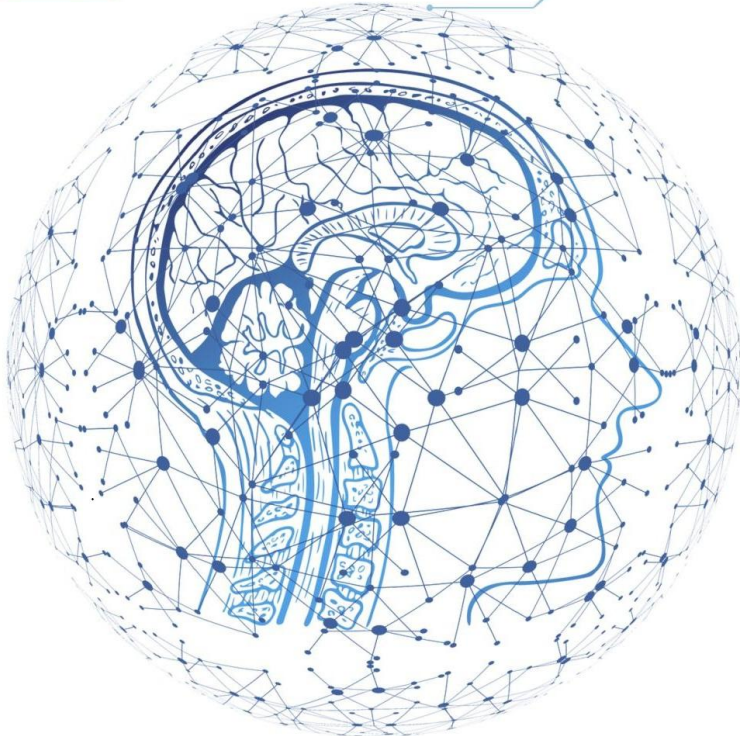
- Gọi:
 - `model.predict(x_test)`
- Kết quả là một xác suất
- Diễn giải:
 - gần 1 → review tích cực
 - gần 0 → review tiêu cực
- Prediction không phải “nhấn tuyệt đối”, mà là mức độ tin cậy

Tổng kết bài học từ IMDB

- Dữ liệu text có thể chuyển thành vector nhị phân
- **Dense + relu** xử lý tốt vector data
- Binary classification:
 - Output: **Dense(1, sigmoid)**
 - Loss: **binary_crossentropy**
- Luôn theo dõi validation để phát hiện overfitting
- **rmsprop** thường là lựa chọn khởi đầu tốt

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Phân loại đa lớp (Multiclass Classification)

Phân loại bản tin Reuters

- Đây là ví dụ phân loại đa lớp
- Bài toán: phân loại bản tin Reuters
- Số lớp: 46
- Mỗi mẫu chỉ thuộc **một** chủ đề
- Đây là bài toán **single-label multiclass classification**

Reuters dataset

- Tập bản tin ngắn được Reuters công bố năm 1986
- Mỗi mẫu là một đoạn tin tức ngắn
- Nhãn là số nguyên từ 0 đến 45
- Tổng cộng 46 lớp chủ đề

Tiền xử lý bản tin Reuters

- Đầu vào được vector hóa giống IMDB
- Mỗi bản tin $\rightarrow$ vector 10,000 chiều
- Nhãn có 2 cách biểu diễn:
 - **One-hot encoding**
 - **Integer labels**
- Trong ví dụ chính, sách dùng **one-hot encoding**

Kiến trúc mô hình phân loại bản tin Reuters

- Mô hình:
 - Dense(64, relu)
 - Dense(64, relu)
 - Dense(46, softmax)
- Vì sao output là 46?
 - vì có 46 lớp
- Vì sao dùng softmax?
 - để tạo phân phối xác suất trên toàn bộ các lớp
 - tổng các xác suất bằng 1

Compile cho multiclass classification

- Optimizer: `rmsprop`
- Loss: `categorical_crossentropy`
- Metric: `accuracy`
- Vì sao dùng `categorical_crossentropy`?
 - đo khoảng cách giữa phân phối xác suất dự đoán và phân phối nhãn thật

Multiclass, one-hot labels → softmax + categorical_crossentropy

Validation và hiện tượng overfitting trên Reuters

- Tách 1,000 mẫu làm validation
- Train 20 epochs
- Quan sát đồ thị:
 - loss train giảm đều
 - val loss giảm rồi tăng
 - val accuracy đạt đỉnh khoảng epoch 9
- Mô hình bắt đầu overfit sau **9 epochs**

Kết quả và baseline của Reuters

- Sau khi train lại 9 epochs, mô hình đạt khoảng **80% accuracy** trên test set
- Random baseline chỉ khoảng **19%**
- Nhận xét:
 - 80% là khá tốt với 46 lớp
 - cần so với baseline để đánh giá đúng ý nghĩa kết quả

Dự đoán và cách đọc output softmax

- `predict()` trả về vector xác suất dài 46 chiều
- Mỗi phần tử là xác suất thuộc một lớp
- Tổng tất cả xác suất = 1
- Lớp dự đoán là: `np.argmax(predictions[i])`

Cách khác để xử lý nhãn

- Không nhất thiết phải one-hot labels
- Có thể giữ nhãn ở dạng số nguyên:
 - `y_train = np.array(train_labels)`
- Khi đó dùng loss:
 - `sparse_categorical_crossentropy`
- Về bản chất toán học:
 - tương đương với `categorical_crossentropy`
 - chỉ khác cách đưa nhãn vào

Information bottleneck trong bài toán nhiều lớp

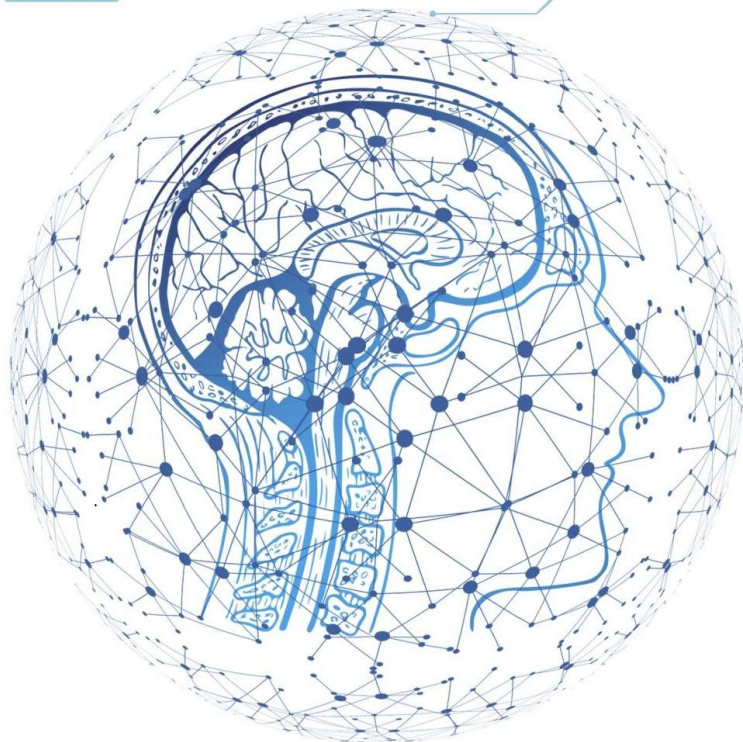
- Nếu tầng trung gian quá nhỏ, mô hình sẽ mất thông tin
- Ví dụ:
 - thêm tầng 4 units trước output 46 lớp
- Kết quả:
 - validation accuracy giảm còn khoảng **71%**
- Vì sao?
 - mô hình bị ép nén quá nhiều thông tin vào không gian thấp chiều

Tổng kết bài học từ phân loại bản tin Reuters

- Multiclass classification:
 - Output layer có N units
 - Activation là **softmax**
- Nếu **labels one-hot**:
 - dùng **categorical_crossentropy**
- Nếu **labels integer**:
 - dùng **sparse_categorical_crossentropy**
- Tránh intermediate layers quá nhỏ khi số lớp lớn

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Hồi quy giá nhà (Regression)

Dự đoán giá nhà

- Đây là ví dụ về hồi quy
- Mục tiêu: dự đoán giá nhà theo dữ liệu khu vực
- Output: một số thực liên tục
- Đây không còn là classification
- Đây là **scalar regression**

Boston Housing dataset

- Dữ liệu gồm:
 - 404 mẫu train
 - 102 mẫu test
- Mỗi mẫu có **13 đặc trưng số**
- Ví dụ đặc trưng:
 - tỉ lệ tội phạm
 - thuế tài sản
 - số phòng trung bình
 - mức độ tiếp cận đường cao tốc
- Target:
 - giá nhà trung vị, đơn vị nghìn USD

Vì sao phải chuẩn hóa dữ liệu?

- Các đặc trưng có giá trị ở những khoảng rất khác nhau
- Nếu không chuẩn hóa:
 - việc học trở nên khó khăn hơn
 - feature lớn có thể lấn át feature nhỏ
- Cách làm:
 - trừ mean của từng feature
 - chia cho standard deviation của từng feature
- Lưu ý:
 - mean và std phải tính từ **training data**

Kiến trúc mô hình cho regression

- Mô hình nhỏ:
 - Dense(64, relu)
 - Dense(64, relu)
 - Dense(1)
- Tại sao output không dùng activation?
 - để mô hình dự đoán được mọi giá trị thực
- Nếu dùng sigmoid:
 - output bị ép vào $[0, 1]$, không phù hợp

Scalar regression → Dense(1) tuyến tính, không activation

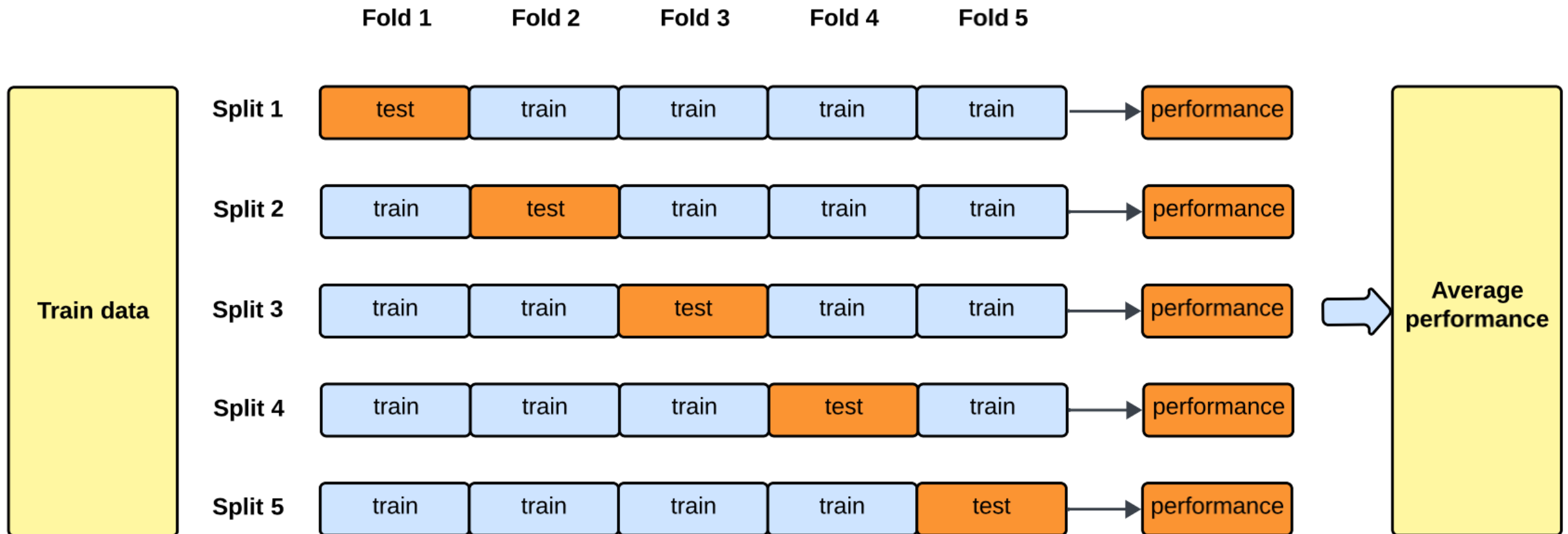
Loss và metric cho regression

- Loss: $mse = \text{mean squared error}$
- Metric: $mae = \text{mean absolute error}$
- Ý nghĩa:
 - MSE phạt mạnh sai số lớn
 - MAE dễ diễn giải hơn
- Ví dụ:
 - $MAE = 0.5$ nghĩa là lệch trung bình khoảng 500 USD

Vì sao cần K-fold validation?

- Dữ liệu quá ít:
 - nếu tách validation đơn giản, tập val rất nhỏ
- Kết quả validation có thể dao động mạnh tùy cách chia
- Giải pháp:
 - **K-fold validation**
- Quy trình:
 - chia dữ liệu thành K phần
 - lần lượt lấy 1 phần làm validation
 - K-1 phần còn lại để train
 - lấy trung bình kết quả

K-fold validation



Chọn số epoch bằng đường cong Validation MAE

- Lưu MAE theo từng epoch ở mỗi fold
- Tính trung bình MAE qua các fold
- Quan sát đồ thị:
 - validation MAE giảm dần lúc đầu
 - sau khoảng **120–140 epochs** thì cải thiện không đáng kể
 - tiếp tục train dễ overfit

Huấn luyện mô hình cuối cùng và kết quả

- Train mô hình cuối với:
 - toàn bộ training data
 - khoảng 130 epochs
 - batch size 16
- Đánh giá trên test set:
 - test MAE khoảng **2.46**
- Diễn giải:
 - dự đoán lệch trung bình khoảng **2,500 USD**

Dự đoán trên dữ liệu mới trong regression

- `predict()` trả về một giá trị thực
- Ví dụ: `array([9.990133], dtype=float32)`
- Diễn giải:
 - nhà đầu tiên trong test được dự đoán khoảng 10,000 USD
- Khác với classification:
 - không có xác suất lớp
 - chỉ có giá trị dự đoán liên tục

Tổng kết bài học từ Boston Housing

- Regression dùng loss khác classification. Thường dùng:
 - **mse** cho loss
 - **mae** cho metric
- Feature có thang đo khác nhau → phải chuẩn hóa
- Dữ liệu ít → nên dùng **K-fold validation**
- Dữ liệu ít → mô hình nên **nhỏ** để tránh overfitting

Tổng kết chương

Bài toán	Loại	Output layer	Activation	Loss	Metric chính
IMDB	Binary classification	Dense(1)	Sigmoid	Binary crossentropy	Accuracy
Reuters	Multiclass classification	Dense(46)	Softmax	Categorical / Sparse categorical crossentropy	Accuracy
Boston Housing	Scalar regression	Dense(1)	Không dùng	MSE	MAE

Tổng kết chương

- Neural network không chỉ là xây layer, mà còn là:
 - tiền xử lý dữ liệu
 - chọn loss đúng
 - chọn output đúng
 - đánh giá đúng
- **Overfitting** là hiện tượng tất yếu cần quan sát bằng **validation**
- Khi dữ liệu ít:
 - dùng mô hình nhỏ
 - dùng K-fold validation
- Khi số lớp lớn:
 - tránh bottleneck ở tầng ẩn

Câu hỏi thảo luận

1. Vì sao binary classification dùng sigmoid còn multiclass classification dùng softmax?
2. Tại sao không nên đánh giá mô hình trên chính training set?
3. Khi nào nên dùng `categorical_crossentropy`, khi nào dùng `sparse_categorical_crossentropy`?
4. Vì sao regression không dùng accuracy?
5. K-fold validation phù hợp trong tình huống nào?
6. Tại sao mô hình quá nhỏ có thể gây information bottleneck?

